TP2

October 1, 2025

1 TP 2 - Régression linéaire

M2 IWOCS, Apprentissage Automatique

1.1 Exercice 1 - Régression linéaire univarié

Les données utilisées pour cet exercice décrivent les tailles d'enfants d'âge compris entre 2 et 8 ans : le fichier `ex1x.dat` correspond à leur âge et `ex1y.dat` correspond à leur taille (en mètres). On a dans ces 2 fichiers, les données de 50 enfants rangées dans le même ordre.

Ces données constituent des exemples d'apprentissage qui vont être utilisées afin de construire un modèle de régression linéaire qui a pour objectif de prédire la taille d'un enfant à partir de son âge.

1. Tout d'abord, nous allons utiliser la fonction `loadtxt`du package `numpy`en Python (ici désigné par `np`). Cette fonction permet de lire les fichiers et de les concertir en vecteurs/matrices :

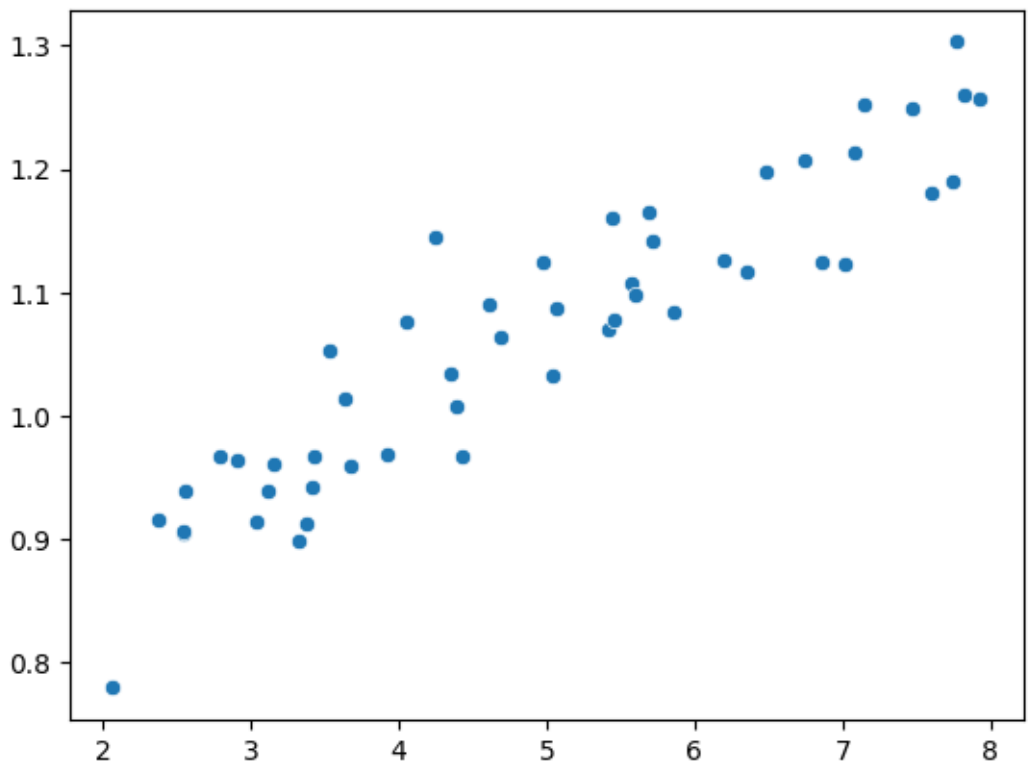
```
x=np.loadtxt('ex1x.dat');
y=np.loadtxt('ex1y.dat');
```

[358]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random
import seaborn as sns
from sklearn.preprocessing import StandardScaler
import pprint
x = np.loadtxt('ex1x.dat')
y = np.loadtxt('ex1y.dat')
```

2. Afficher le nuage de points (x_i, y_i).

[359]:
```
sns.scatterplot(x=x, y=y)
```

[359]: <Axes: >



3. Définir en Python, la fonction hypothèse correspondant à un modèle de régression linéaire. Soit le vecteur $\theta = (\theta_0, \theta_1)$, cette fonction hypothèse s'écrit

$$h(\theta_0, \theta_1, x) = \theta_0 + \theta_1 x$$

[360]:
```
def h(teta0, teta1, x):
    return teta0 + teta1 * x
```

4. Définir en Python, la fonction de coût $J(\theta_0, \theta_1)$ telle que vue en cours.

[361]:
```
def J(teta0, teta1):
    tampon = 0
    m = len(x)
    for i in range(m):
        tampon += (h(teta0, teta1, x[i]) - y[i])**2
    return 1/(2*m) * tampon
```

5. Définir en Python, une fonction qui réalise une itération et qui va renvoyer θ_0^* et θ_1^*, en fonction de θ_0 et θ_1, selon les formules de descente de gradient vue en cours.
Pour faire fonctionner cette méthode de gradient, il faut définir la valeur du coefficient d'apprentissage noté α dans le cours ; il régle la profondeur de descente. On propose ici que ce coefficient prenne une valeur constante égale à 0,07. On partira aussi des valeurs initiales $\theta_0 = \theta_1 = 0$.

[362]:
```
def deltaJFonc(teta0, teta1):
    m = len(x)
    dJ0 = 0
    dJ1 = 0
    for i in range(m):
        erreur = h(teta0, teta1, x[i]) - y[i]
        dJ0 += erreur
        dJ1 += erreur * x[i]
    return (1/m * dJ0, 1/m * dJ1)

def gradient(teta0, teta1):
    alpha = 0.07
    deltaJ = deltaJFonc(teta0, teta1)
    teta0Star = teta0 - alpha * deltaJ[0]
    teta1Star = teta1 - alpha * deltaJ[1]
    return (teta0Star, teta1Star)
```

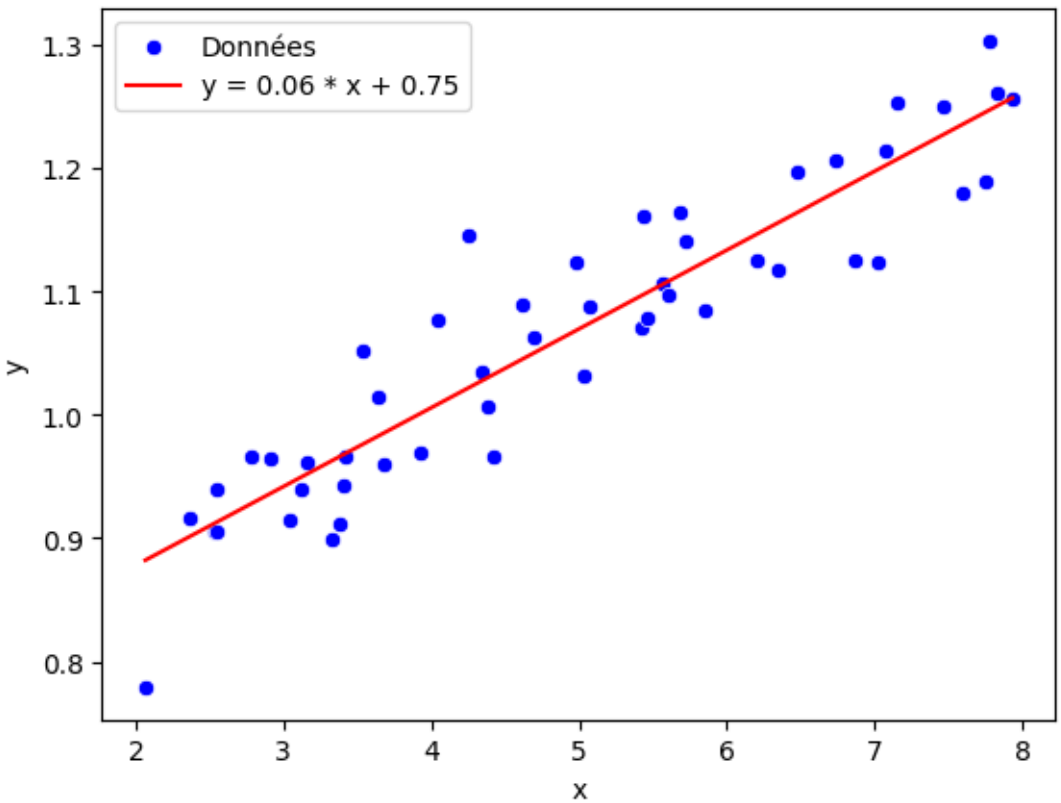
6. Faire tourner la méthode de descente de gradient sur quelques itérations puis représenter la droite

$$y = \theta_0 + \theta_1 x$$

sur le nuage de points.

[363]:
```
teta = [0, 0]
for i in range(500000):
    teta = gradient(teta[0], teta[1])

df = pd.DataFrame({'x': x, 'y': y})
sns.scatterplot(data=df, x='x', y='y', color='blue', label='Données')
x_line = np.linspace(min(x), max(x), 100)
y_line = teta[0] + teta[1] * x_line
sns.lineplot(x=x_line, y=y_line, color='red', label=f'y = {teta[1]:.2f} * x +␣
↪{teta[0]:.2f}')
```

[363]: <Axes: xlabel='x', ylabel='y'>



7. Faire tourner la méthode de descente de gradient pendant toutes les itérations nécessaires à faire converger la solution θ recherchée. Pour cela on définit le critère d'arrêt du processus itératif par

$$\left|\frac{J(\theta^*) - J(\theta)}{J(\theta)}\right| < 10^{-3}$$

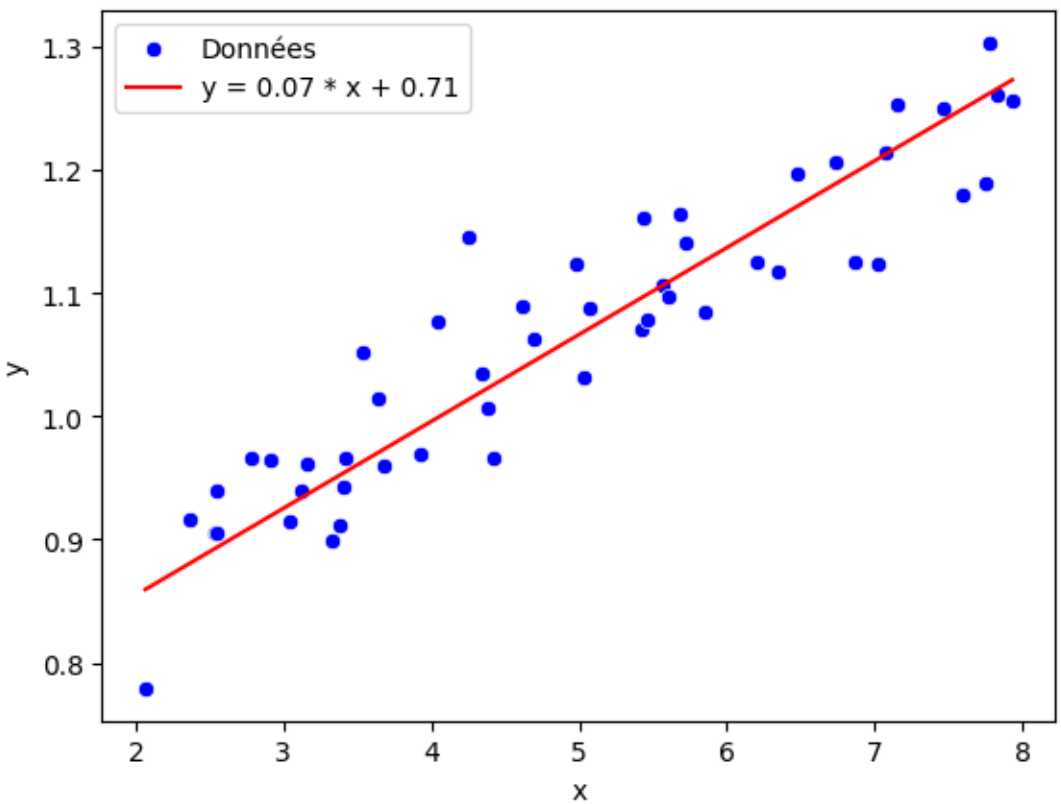
Afficher la droite obtenue suite à cette convergence sur le nuage de points.

[364]:
```
teta = [0, 0]
valeurI = 0
for i in range(500000):
    tetaBefore = [teta[0], teta[1]]
    teta = gradient(teta[0], teta[1])
    valeurI = i
    if(abs((J(teta[0], teta[1]) - J(tetaBefore[0], tetaBefore[1])) /␣
 ↪J(tetaBefore[0], tetaBefore[1])) < 10**-3):
        break

print(f"Nombre d'itérations : {valeurI}.")
df = pd.DataFrame({'x': x, 'y': y})
sns.scatterplot(data=df, x='x', y='y', color='blue', label='Données')
xline = np.linspace(min(x), max(x), 100)
```

```
yline = teta[0] + teta[1] * x_line
sns.lineplot(x=xline, y=yline, color='red', label=f'y = {teta[1]:.2f} * x +␣
↪{teta[0]:.2f}')
```

```
Nombre d'itérations : 406.
```

[364]: <Axes: xlabel='x', ylabel='y'>



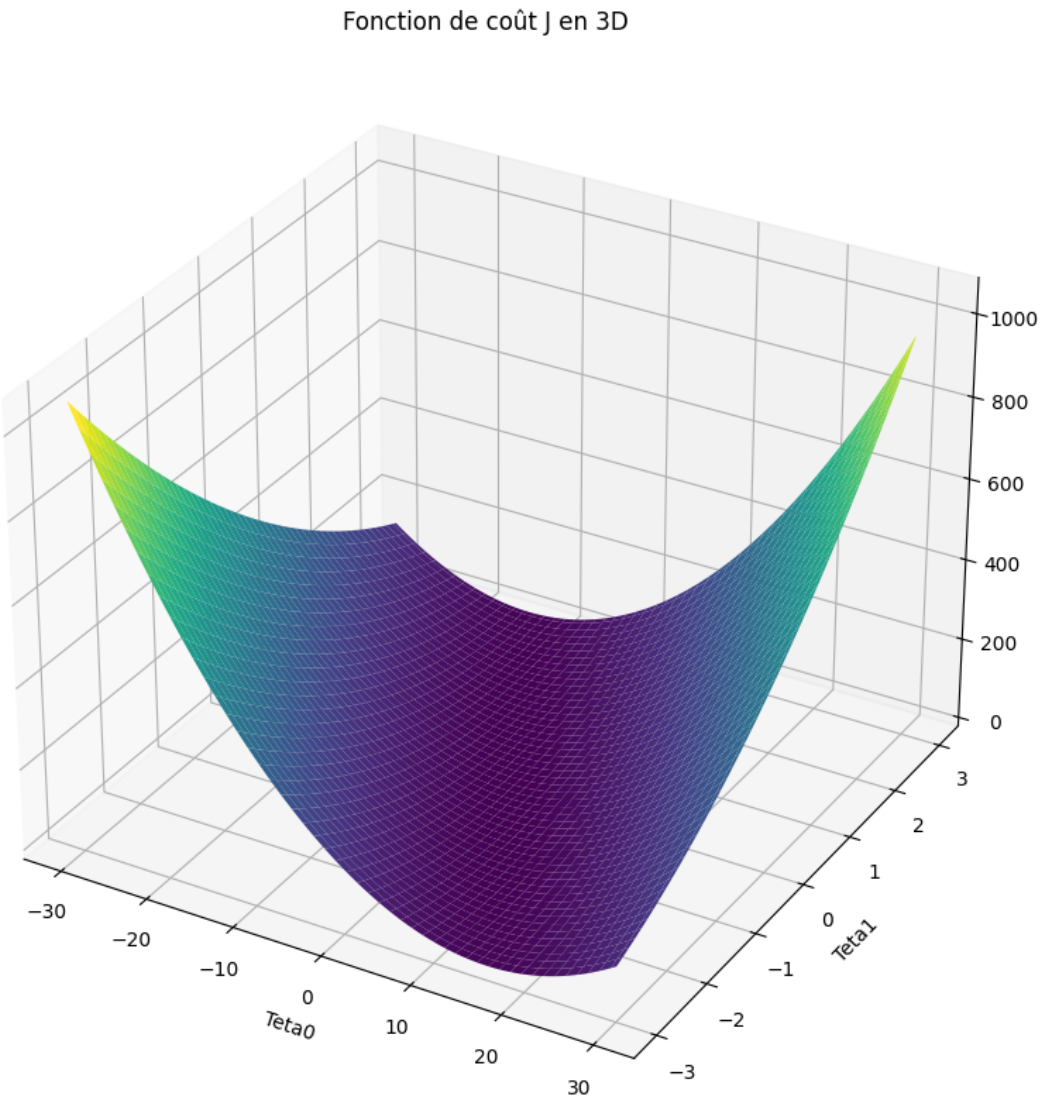
8. On peut désormais utiliser le modèle pour faire des prédictions : quelle est la taille de 3 enfants d'âges respectifs 3, 5 et 7 ans ?

[365]:
```
print(f"Taille des enfants de 3 ans : {(teta[0] + teta[1] * 3):.2f}m.")
print(f"Taille des enfants de 5 ans : {(teta[0] + teta[1] * 5):.2f}m.")
print(f"Taille des enfants de 7 ans : {(teta[0] + teta[1] * 7):.2f}m.")
```

```
Taille des enfants de 3 ans : 0.93m.
Taille des enfants de 5 ans : 1.07m.
Taille des enfants de 7 ans : 1.21m.
```

9. Nous allons maintenant visualiser la fonction de coût $J(\theta)$ en 3D sur une grille de base 100 x 100 et des valeurs pour θ dans les intervalles suivants (il faudra donc prendre 100 valeurs réparties dans ces intervalles) : $\theta_0 \in [-30; 30]$ et $\theta_1 \in [-3; 3]$.

[366]:
```
listOfTeta0 = np.linspace(-30, 30, 100)
listOfTeta1 = np.linspace(-3, 3, 100)

theta0, theta1 = np.meshgrid(listOfTeta0, listOfTeta1)
J_vals = np.zeros_like(theta0)

for i in range(theta0.shape[0]):
    for j in range(theta1.shape[1]):
        J_vals[i, j] = J(theta0[i, j], theta1[i, j])

fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(theta0, theta1, J_vals, cmap='viridis', edgecolor='none')
ax.set_xlabel('Teta0')
ax.set_ylabel('Teta1')
ax.set_title('Fonction de coût J en 3D')
plt.show()
```



1.2 Exercice 2 - Régression linéaire multivariée et écriture matriciellle de la méthode de descente du gradient

Dans ce problème, on utilise des données correspondant à 47 exemples d'apprentissage sur des données immobilières à Portland, Oregon (USA). Les données d'entrée x (stockées dans le ficheir `ex2x.dat`) correspondent aux surfaces et au nombre de pièces de ces 47 appartements et la donnée cible y (stockée dans le fichier `ex2y.dat`) correspond au prix de ces mêmes appartements.

1. Utiliser la fonction `loadtxt`du package `numpy`en Python (ici désigné par `np`). Cette fonction permet de lire les fichiers et de les concertir en vecteurs/matrices :

```
x=np.loadtxt('ex2x.dat');
y=np.loadtxt('ex2y.dat');
```

[397]:
```
X = np.loadtxt('ex2x.dat')
Y = np.loadtxt('ex2y.dat')
```

2. Prétraitement des données : utiliser la fonction `sklearn.preprocessing.StandardScaler` pour normaliser les données.

[398]:
```
standardScalerX = StandardScaler()
xNormalize = standardScalerX.fit_transform(X)
m = len(Y)
X = np.column_stack([np.ones(m), xNormalize])
Y = Y.reshape(-1, 1)
```

3. Définir les fonctions en Python sous forme matricielle pour représenter la fonction hypothèse $h_\theta(X)$, le vecteur tel que défini dans le cours :$E = h_\theta(X) - Y$, la fonction décrivant une itération et la fonction de coût $J(\theta)$.

[399]:
```
def hypothese(teta, X):
    return X @ teta

def E(teta, X, Y):
    return hypothese(teta, X) - Y

def cout(teta, X, Y):
    m = len(Y)
    vE = E(teta, X, Y)
    return (1 / (2 * m)) * (vE.T @ vE)

def iteration(teta, X, Y, alpha):
    m = len(Y)
    vE = E(teta, X, Y)
    newTeta = teta - (alpha / m) * (X.T @ vE)
    return newTeta
```

4. Pour la valeur du taux d'apprentissage $\alpha = 0,07$, effectuer le calcul de régression permettant d'obtenir le vecteur $\theta = (\theta_0, \theta_1, \theta_2)$ optimal permettant de calculer la meilleure régression linéaire multivariée sur le jeu de données d'apprentissage.

[400]:
```
alpha = 0.07
nbIters = 1000
epsilon = 1e-6

teta = np.zeros((3, 1))
for i in range(nbIters):
    coutNow = cout(teta, X, Y)
    newTeta = iteration(teta, X, Y, alpha)

    if abs((cout(newTeta, X, Y) - coutNow) / coutNow)[0][0] < epsilon:
        print(f"Après {i} itérations on a convergé !")
        break
```

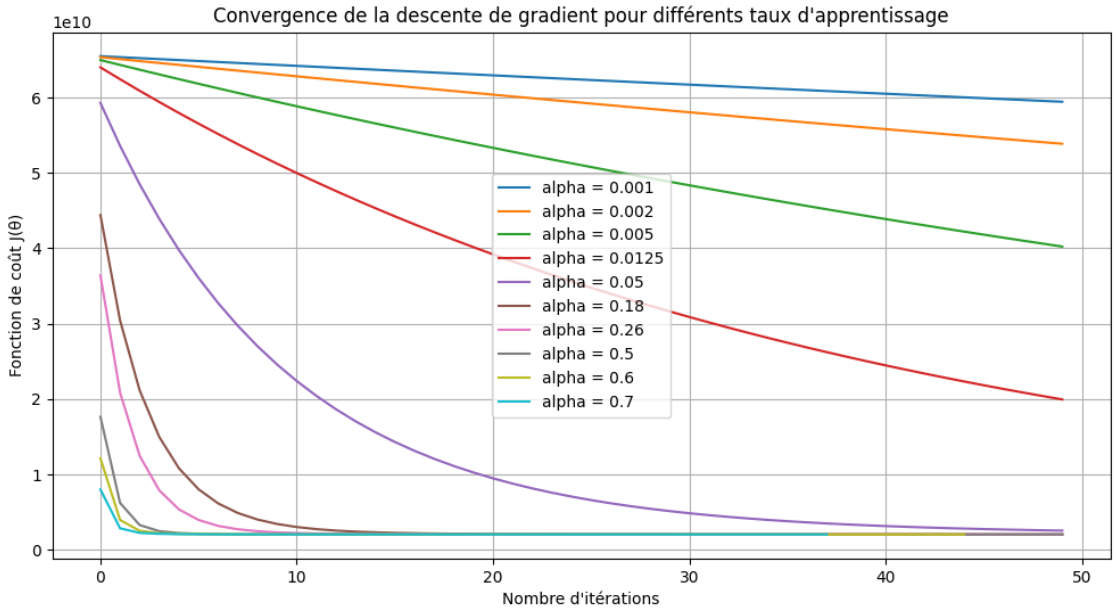
```
    teta = newTeta

print(f"teta optimal : {[float(teta[x][0]) for x in range(3)]}")
```

```
Après 171 itérations on a convergé !
teta optimal : [340411.2709337188, 109172.34662694007, -6302.9052774667125]
```

5. Nous allons maintenant automatiser la recherche du meilleur taux d'apprentissage $\alpha \in [0,001; 10]$. Pour ce faire, on devra calculer pour chaque itération la valeur de la fonction de coût $J(\theta)$ et on stockera toutes ces valeurs dans un vecteur. Comme on veut sélectionner un taux d'apprentissage efficace, on va comparer les résultats de calcul de $J(\theta)$ sur 50 itérations en changenat de taux d'apprentissage à chaque série d'itérations. Les valeurs de ce taux doivent rester dans l'intervalle initialement donné $\alpha \in [0,001; 10]$. On tracera alors les courbes représentant en abscisse, le nombre d'itérations et en ordonnées les valeurs de la fonction de coût ; chaque courbe correspond à une valeur du taux d'apprentissage. A partir de ces courbes, sélectionner ce qui parît être le meilleur taux d'apprentissage et recalculer le vecteur θ jusqu'à la convergence. Utiliser ce vecteur θ pour prédire le prix d'un logement de 1650 m2 et de 3 pièces.

[401]:
```
alphas = [0.001, 0.002, 0.005, 0.0125, 0.05, 0.18, 0.26, 0.5, 0.6, 0.7]
nbIters = 50
epsilon = 1e-12
resultats = {alpha:[] for alpha in alphas}
for alpha in alphas:
    teta = np.zeros((3, 1))
    for i in range(nbIters):
        coutNow = cout(teta, X, Y)
        newTeta = iteration(teta, X, Y, alpha)

        resultats[alpha].append(cout(newTeta,X,Y)[0][0])

        if abs((cout(newTeta, X, Y) - coutNow) / coutNow)[0][0] < epsilon:
            break

        teta = newTeta

plt.figure(figsize=(12, 6))
for alpha in alphas:
    plt.plot(range(len(resultats[alpha])), resultats[alpha], label=f'alpha =␣
 ↪{alpha}')

plt.xlabel('Nombre d\'itérations')
plt.ylabel('Fonction de coût J( )')
plt.title('Convergence de la descente de gradient pour différents taux␣
 ↪d\'apprentissage')
plt.legend()
plt.grid(True)
```

```
plt.show()
```



À partir de ces courbes, je décide de prendre alpha = 0.18.

[403]:
```
alpha = 0.18
nbIters = 1000
epsilon = 1e-12

teta = np.zeros((3, 1))

for i in range(nbIters):
    coutNow = cout(teta, X, Y)
    newTeta = iteration(teta, X, Y, alpha)
    if abs((cout(newTeta, X, Y) - coutNow) / coutNow)[0][0] < epsilon:
        break
    teta = newTeta

print(f"teta convergé : {[float(teta[x][0]) for x in range(3)]}")

print("Prédiction du prix pour un logement d'une superficie de 1650 m2 et␣
 ↪disposant de 3 pièces.")

xDataPred = np.array([
    [1650, 3],
])

xDataPred_norm = standardScalerX.transform(xDataPred)
```

```python
xDataPred_norm = np.column_stack([np.ones(1), xDataPred_norm])
prediction = hypothese(teta, xDataPred_norm)
print(f"{prediction[0][0]:.0f}€")
```

```
teta convergé : [340412.6595744532, 109447.63476686511, -6578.193151384572]
Prédiction du prix pour un logement d'une superficie de 1650 m2 et disposant de
3 pièces.
293081€
```